Reflection: The Time Traveller - 1995 Video Rental System

Italo de Moura Santos - MSc Data Science - Midterm Project

Object-Oriented Design

I modeled the video store with three classes, each with a single, clear responsibility. ‘Movie’ stores the data of a title (title, genre, release year) and its current state through the ‘is_rented’ flag. ‘Customer’ represents a person and keeps the list of movies that person has rented. ‘VideoStore’ acts as the coordinator: it owns the catalog and the customers, and it contains all the rental rules (renting, returning, searching and displaying).

This separation relies on three OOP ideas. Encapsulation: each object manages its own data, and the rest of the program interacts with it through well-defined methods. Single responsibility: a change in the rental rules only touches ‘VideoStore’, while ‘Movie’ and ‘Customer’ stay simple. Composition: the store "has" movies and customers, and a customer "has" rented movies, which matches how the real-world problem works. I used the special methods ‘__init__’ to give every object a valid initial state and ‘__str__’ to define in one place how an object is displayed, which keeps the display methods short and consistent. Inheritance was not needed here, because the three entities are different concepts rather than variations of one another, and forcing a hierarchy would only add complexity.

Data Structures

In ‘VideoStore’ I used dictionaries: title -> ‘Movie’ and name -> ‘Customer’. Titles and names act as unique keys, so a lookup is O(1) on average instead of O(n) with a list, and detecting duplicates is trivial (if title in self.movies). In ‘Customer.rented_movies’ I used a list, because a customer holds a small, ordered collection that I mainly iterate over, and no key-based access is needed. Movies are never removed from the catalog when rented, the ‘is_rented’ flag changes instead, so the store can always display both the full catalog and the rented movies.

Error Handling

Invalid operations raise exceptions instead of printing messages inside the classes. ‘KeyError’ is used when a movie or customer does not exist, and ‘ValueError’ when the operation is not valid in the current state (movie already rented, duplicate customer, returning a movie the customer did not rent). This keeps the logic separate from the user interface and lets the caller decide how to handle each case.

Limitations and Improvements

The system assumes one copy per title and uses titles and names as keys, so two customers with the same name would conflict. With more time I would add unique IDs, multiple copies per title, rental dates with due dates and late fees, a return history and persistence (JSON file or database).